

Matemática del Entorno AG-RACE y Dinámica del Agente

Proyecto Jackpot Jockeys

11 de marzo de 2026

1. Introducción

Este documento describe el modelo matemático que rige las físicas del vehículo (*Hovercraft*) y las observaciones que el agente de Inteligencia Artificial (IA) realiza dentro del entorno **AG-RACE**. El sistema está diseñado tanto para ser evaluado por heurísticas matemáticas (*Pure Pursuit*) como para entrenar políticas de Aprendizaje por Refuerzo (RL).

2. Fundamentos Matemáticos y Determinismo

Para garantizar una evaluación justa y reproducible durante el entrenamiento RL, el entorno AG-RACE exige **determinismo absoluto**. Todos los cálculos espaciales y pseudoaleatorios se generan a partir de una *Semilla Inicial* (S_0), garantizando que dos episodios ejecutados bajo la misma secuencia de acciones produzcan idénticas observaciones y recompensas.

2.1. Generación Procedural de Pistas (Splines Hermite)

La curva central paramétrica de la pista $C(s)$ se genera interpolando N puntos de control generados aleatoriamente dentro de rangos cilíndricos controlados por la semilla determinista. La interpolación se modela usando splines cúbicos de tipo Hermite, donde el segmento $P_i(u)$ para $u \in [0, 1]$ se define como:

$$P_i(u) = (2u^3 - 3u^2 + 1)P_0 + (u^3 - 2u^2 + u)M_0 + (-2u^3 + 3u^2)P_1 + (u^3 - u^2)M_1$$

Donde P_0, P_1 son puntos de control y M_0, M_1 son sus respectivas tangentes parametrizadas.

3. Dinámica del Vehículo y Motor Físico Jolt

El vehículo se modela en un espacio bidimensional (2D) sujeto a aceleraciones lineales y atenuación de derrapes.

3.1. Estado del Vehículo y Vectores

El estado dinámico del vehículo se define por su posición global $\mathbf{p}$, su velocidad lineal $\mathbf{v}$ y su rotación θ . Los vectores direccionales frontales y laterales se definen como:

$$\hat{\mathbf{f}} = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}, \quad \hat{\mathbf{r}} = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}$$

3.2. Actualización de Movimiento y Giro

El agente proporciona tres valores escalares de acción: Aceleración ($T \in [0, 1]$), Freno ($B \in [0, 1]$) y Giro ($S \in [-1, 1]$). La orientación y la aceleración se actualizan discretamente:

$$\theta_{t+\Delta t} = \theta_t + (S \cdot \omega_{turn} \cdot \Delta t)$$

$$\mathbf{v}_{t+\Delta t} = \mathbf{v}_t + \left(\hat{\mathbf{f}} \cdot T \cdot a_{max} \cdot \mu_{fric} \cdot \Delta t \right)$$

3.3. Derrape (Drift) y Velocidad Transversal

Para simular el efecto *Hovercraft*, se descompone la velocidad en frontal (v_f) y lateral (v_l):

$$v_f = \mathbf{v} \cdot \hat{\mathbf{f}} \quad \text{y} \quad v_l = \mathbf{v} \cdot \hat{\mathbf{r}}$$

La velocidad lateral se atenúa mediante un coeficiente K_{grip} sujeto a deterioro ambiental μ_{drift} :

$$v_{l,new} = v_l - \text{sgn}(v_l) \cdot \min(|v_l|, K_{grip} \cdot \mu_{drift} \cdot |v_l| \cdot \Delta t)$$

4. Métricas del Entorno y Recompensas (RL)

4.1. Progreso en la Pista

El progreso del agente, $s \in [0, 1]$, se calcula normalizando el avance sobre la longitud de la spline L : El recompensa por paso t es la del diferencial transcurrido:

$$R_{avance} = (s_t - s_{t-1}) \cdot L$$

4.2. Error Lateral (Track Distance) y Penalización

Para determinar si el vehículo está en la pista, calculamos la distancia eucídea residual hacia el centro de la spline $C(s)$:

$$err_{lateral} = \|\mathbf{p} - C(s_{closest})\| - \frac{TrackWidth}{2}$$

Si $err_{lateral} > 0$, el agente sale de la pista. Se le aplica una severa penalización proporcional para obligarlo a regresar:

$$R_{off_track} = -\alpha \cdot e^{\beta \cdot err_{lateral}}$$

5. Condiciones Variables y Perturbaciones (World Events)

El entorno dinámico inyecta fallos y ruidos matemáticos (World Events) para entrenar la generalización del agente:

5.1. Evento: LOW_GRIP (Baja Adherencia)

Al desencadenarse, la fricción base disminuye proporcional a la *severidad* del evento:

$$\mu_{nuevo}(t_0) = \mu_{base} \cdot (1,0 - severity)$$

El recupero de tracción es progresivo y sigue una función lineal por cada Δt :

$$\mu_{drift}(t) = \min(\mu_{base}, \mu_{drift}(t - \Delta t) + recovery_{rate} \cdot \Delta t)$$

6. Arquitectura Neuronal y Neuroevolución

Para resolver este proceso de toma de decisiones no estacionario se emplea una política neuronal compacta de unos 12,000 parámetros entrenada mediante Neuroevolución (ES/ARS o CMA-ES), lo que permite optimizar parámetros al tiempo que se evitan discontinuidades causadas por caídas o salidas de pista abruptas.

6.1. Estructura de Observaciones (85 Dimensiones)

El vector estocástico que recibe el cerebro comprende:

- **Sensores de Pista (33 dim):** 31 raycasts polarizados normalizados y métricas geométricas (e_θ , $err_{lateral}$).
- **Estado Propio (10 dim):** Velocidad, aceleración, rumbo polarizado $\sin / \cos(\theta)$, y estado de derrape.
- **Rivales (18 dim):** Pooling de características para 3 rivales usando métricas relativas (distancia, velocidad relativa, ángulos).
- **Eventos y Condiciones (24 dim):** Inyecciones condicionales de variables climáticas, *embeddings* parametrizados de *hazards* en pista y duraciones de los *debuffs* sufridos.

6.2. Política Multi-Habilidad (Multi-Skill Policy)

La red neuronal procesa los *embeddings* de los eventos de forma individual para evitar sobredimensionar la red (one-hot vectors). El *pool* de rivales se unifica matemáticamente utilizando *Max-Pooling*. El corazón de la red se diseña con una **arquitectura bicéfala** que consta de:

- Un tronco compartido con capas tipo *tanh*.
- Una cabeza entrenada enfocada a la conducción en solitario (Driving: $\mathbf{u}^{(d)}$).
- Una cabeza enfocada al adelantamiento defensivo u ofensivo (Overtaking: $\mathbf{u}^{(o)}$).

Se emplea una compuerta sigmoideal (*Gate Controller* g) calculada sobre las distancias hacia los rivales para mezclar la salida de forma suave:

$$\mathbf{u} = (1 - g) \cdot \mathbf{u}^{(d)} + g \cdot \mathbf{u}^{(o)}$$

6.3. Fitness Multi-objetivo y Varianza

La función de *fitness* para los métodos CMA-ES engloba un *Trade-off* entre competitividad y seguridad, combinando el avance por tick (Δs_t) y variables penalizantes (colisión, off-track, stuck, zig-zag).

Crucialmente, para fomentar la generalización y combatir las trampas procedurales que causan fallos catastróficos, se impone un término estabilizador de **varianza inter-seed** en el objetivo esperado:

$$J(\theta) = \mathbb{E}_\omega [\dots - \lambda_v \text{Var}_\omega(R_\omega(\theta))]$$